

Université Ibn Tofaïl

Faculté des Sciences de Kenitra

Master : Big Data, Intelligence Artificielle et Applications Avancées

Rapport du Mini Projet

Sous le thème :

UtilityHub : Système Modulaire de Gestion d'Outils Utilitaires

Présenté par : Salimi Achraf

Encadré par : Dr. Hatim Derrouz

Année universitaire 2025/2026

Remerciements

Nous tenons à exprimer notre profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation du projet **UtilityHub**.

Nos remerciements s'adressent tout d'abord à notre professeur du module, pour nous avoir proposé ce mini-projet stimulant et pour ses précieux conseils qui ont guidé notre travail tout au long du développement de cette application modulaire.

Nous tenons également à remercier l'ensemble du corps professoral de l'**UIT** pour la qualité de l'enseignement et l'accompagnement pédagogique qui nous ont permis d'acquérir les compétences techniques nécessaires à la réussite de ce projet.

Enfin, nous remercions nos collègues et nos proches pour leurs encouragements et leur soutien moral, qui ont été une source de motivation précieuse durant les différentes phases de conception et de réalisation.

Ce travail est le fruit d'un investissement collectif passionné, et nous sommes reconnaissants envers tous ceux qui ont rendu cette expérience aussi enrichissante.

Sommaire

Remerciements.....	1
Sommaire	2
Introduction.....	3
Objectifs	4
Architecture Générale	5
Conception et Réalisation	7
1. Technologies utilisées	7
1.1.Pandas et fichiers CSV	7
1.2. FileLock (Gestion de la synchronisation).....	7
1.3. Frameworks d'Interface (Desktop & Web).....	7
1.4. Bibliothèques Spécialisées	8
2. Couche Core (Managers)	8
2.1. CsvManager.....	8
2.2. HistoryManager	9
2.3. SettingsManager	11
3. Modules Fonctionnels	12
3.1. Module Convertisseur.....	12
3.2. Module Téléchargeur (Downloader)	18
3.3. Module Notes	22
3.4. Module Todo (Tâches)	26
3.5. Module Historique.....	29
Mise en œuvre Technique	30
1. Interface Desktop (PyQt5)	30
1.1. Navigation et Menu principal	30
1.2. Rapidité et Travail en arrière-plan.....	31
1.3. Le défi du GIL (Global Interpreter Lock)	31
2. Interface Web (Flask).....	31
2.1. Organisation par modules	31
2.2. Suivi en direct du téléchargement	31

2.2.1. Détails techniques du SSE :	32
2.3. Un design moderne et dynamique	32
3. Synchronisation et Sécurité (FileLock)	32
4. Pourquoi le Desktop est plus avancé ?	32
Présentation Graphique	33
1. Interface Desktop (PyQt5)	33
1.1. Module Convertisseur	34
1.2. Module Téléchargeur	34
1.3. Gestion des Notes	35
1.4. Liste de tâches	36
1.5. Historique des activités	37
2. Interface Web (Flask)	38
2.1. Page d'accueil / Convertisseur	39
2.2. Page du Téléchargeur	39
2.3. Page des Notes	39
2.4. Liste des Tâches	40
2.5. Historique	41
Conclusion	42

Introduction

Le projet **UtilityHub** est né d'une initiative personnelle visant à concevoir une application "Multi-Utils" polyvalente, capable de m'assister efficacement dans mes tâches quotidiennes futures. J'ai proposé ce concept avec une double ambition : d'une part, explorer en profondeur le fonctionnement interne d'outils numériques courants (tels que les algorithmes de conversion ou les protocoles de téléchargement média), et d'autre part, mettre en pratique de manière intensive et simultanée les frameworks **PyQt** (Desktop) et **Flask** (Web).

Dans le cadre de ce mini-projet au sein de l'**UIT**, l'objectif était de bâtir une architecture modulaire solide permettant de centraliser quatre fonctionnalités essentielles : la conversion de fichiers, le téléchargement multi-sources, la gestion de notes et le suivi de tâches. Cette approche hybride a permis d'aborder des problématiques concrètes de développement logiciel, notamment la synchronisation de données entre différentes interfaces via **Pandas** et la sécurisation des accès concurrents avec **FileLock**.

Ce rapport technique détaille la démarche de conception adoptée, les solutions technologiques implémentées et le bilan de cette expérience enrichissante combinant ingénierie logicielle et développement web.

Avertissement : Ce projet a été mené strictement à des fins pédagogiques pour étudier les protocoles de streaming vidéo, l'extraction de données et les techniques d'automatisation. Aucun contenu protégé par des droits d'auteur n'a été distribué ou utilisé au-delà de ce qui est légalement autorisé. Toutes les expériences ont été limitées à des données accessibles au public.

Objectifs

L'objectif de ce projet est de créer une application appelée **UtilityHub** qui rassemble quatre outils pratiques dans une seule plateforme :

- **Convertisseur de fichiers** : pour transformer des documents, des images ou de l'audio.
- **Téléchargeur de médias** : pour télécharger des vidéos ou des musiques depuis plusieurs sources.
- **Gestionnaire de notes** : pour écrire et enregistrer des textes rapidement.
- **Liste de tâches (Todo List)** : pour organiser ses activités quotidiennes.

Le projet repose sur plusieurs piliers technologiques :

1. **PyQt5** : pour l'interface logicielle (Desktop).
2. **Flask** : pour l'interface accessible par navigateur (Web).
3. **Pandas & CSV** : pour une gestion simple et légère des données.
4. **Outils spécialisés** : comme **yt-dlp**, **FFmpeg**, **FileLock**, etc

L'application utilise une **architecture modulaire**. Chaque outil est indépendant, ce qui rend le code plus clair et facilite l'ajout de nouvelles fonctions.

Architecture Générale

L'architecture de UtilityHub est organisée en **quatre couches principales** pour assurer une séparation claire entre les données, la logique et les interfaces :

1. **Couche de Persistance (Données)** : Elle comprend les fichiers **CSV** (notes, tâches, historiques) et le fichier **JSON** pour les réglages.
2. **Couche Core** : Composée des **Managers** (CsvManager, SettingsManager, HistoryManager), elle gère l'accès aux données. C'est cette couche qui utilise le **FileLock** pour garantir l'intégrité des données et prévenir les conflits d'accès concurrents entre les interfaces Web et Desktop.
3. **Couche Service** : C'est le cerveau de chaque module (Converter, Downloader, etc.). Ces services sont purement algorithmiques et ne dépendent pas de l'interface graphique. Ils peuvent donc être appelés aussi bien par PyQt5 que par Flask.
4. **Couche Présentation (Interfaces)** : Elle se divise en deux environnements distincts :
 - a. **Desktop (PyQt5)** : Pour une utilisation locale rapide et puissante.
 - b. **Web (Flask)** : Pour un accès flexible via navigateur.

Nous ajoutons également une **Couche d'Outils Externes** qui contient les exécutables nécessaires comme **FFmpeg** et **node.exe**, permettant ainsi à l'application d'être autonome et prête à l'emploi.

```
utilityhub/
app.py
desktop_app.py
assets/
    active_tab_btns/
    inactive_tab_btns/
    red_X_icon.png
    blue_download_icon.png
    ...
tools/
    ffmpeg.exe # Binaire FFmpeg pour fonctionnement autonome
    node.exe # Binaire node
data/
    notes.csv
    todos.csv
    history.csv
    settings.json
    uploads/ # stockage temporaire
    outputs/
        converted/
        downloads/

core/
    csv_manager.py
    history_manager.py
    settings_manager.py

modules/
    converter/
        converter_service.py
        converter_routes.py
    downloader/
        downloader_service.py
        downloader_routes.py
    notes/
        notes_service.py
        notes_routes.py
    todo/
        todo_service.py
        todo_routes.py
web/
    templates/
    static/
desktop/
    main_window.py
    windows/
        converter_window.py
        downloader_window.py
        notes_window.py
        todo_window.py
        history_window.py
```

Conception et Réalisation

1. Technologies utilisées

Pour ce projet, nous avons choisi des technologies spécifiques pour répondre à des besoins précis :

1.1. Pandas et fichiers CSV

Au lieu d'utiliser une base de données complexe (comme SQL), nous avons choisi des fichiers **CSV** manipulés par la bibliothèque **Pandas**.

1.2. FileLock (Gestion de la synchronisation)

Comme nous avons deux interfaces (Desktop et Web) qui peuvent être ouvertes en même temps, il y a un risque de conflit si les deux essaient de modifier le même fichier CSV.

- **Solution : FileLock** crée un petit "verrou" temporaire. Si le Web écrit une note, le Desktop attend son tour, évitant ainsi de corrompre les données.

1.3. Frameworks d'Interface (Desktop & Web)

- **PyQt5** : Le framework principal pour l'application de bureau. Il offre une grande flexibilité pour créer une interface utilisateur (UI) sophistiquée, gérant nativement les événements et le multi-threading (via QThread).
- **Flask** : Utilisé pour la partie serveur web. Ce micro-framework Python permet d'exposer les fonctionnalités du projet via un navigateur, avec une structure modulaire basée sur les "Blueprints".
- **Bootstrap** : Intégré à l'interface web pour assurer un design moderne et professionnel. Il facilite la création d'une mise en page "responsive" et esthétique grâce à ses composants prédéfinis.

1.4. Bibliothèques Spécialisées

Pour offrir des fonctionnalités avancées, nous avons intégré plusieurs outils techniques :

- **yt-dlp** : Utilisé pour extraire et télécharger des vidéos depuis presque n'importe quelle plateforme web.
- **FFmpeg** : Ce binaire professionnel est piloté par Python pour convertir les formats audio et vidéo. L'utilisation de subprocess permet de lancer ces tâches lourdes en arrière-plan.
- **PIL / Pillow** : La bibliothèque de référence pour manipuler les images (redimensionnement, changement de format).
- **vtracer** : Un outil performant qui permet de transformer une image classique (pixels) en une image vectorielle (SVG).
- **xhtml2pdf & Markdown** : Ces outils travaillent ensemble pour transformer des textes simples ou des notes au format Markdown en documents PDF propres et professionnels.

2. Couche Core (Managers)

Le "Cœur" de l'application est composé de trois gestionnaires (Managers) qui centralisent les tâches répétitives pour tous les outils :

2.1. CsvManager

C'est le composant le plus important pour la gestion des données. Il centralise les quatre opérations fondamentales : le **chargement (load)**, la **sauvegarde (save)**, le **verrouillage (lock)** et la **génération d'identifiants (generate id)** pour les trois fichiers CSV centraux (**history.csv**, **notes.csv** et **todos.csv**).

Plus précisément, il s'occupe de :

- **La lecture et l'écriture** : Il manipule les fichiers CSV proprement, garantissant que les données sont traitées de manière cohérente par toutes les parties de l'application.
- **La génération d'IDs uniques** : Chaque nouvelle note ou tâche est marquée d'un identifiant unique généré automatiquement, ce qui permet de les identifier précisément sans risque de confusion.
- **La sécurisation des accès** : C'est ici que le système de verrouillage (**FileLock**) est implémenté pour éviter que le Web et le Desktop ne se gênent lors d'accès simultanés, prévenant ainsi toute corruption de données.

```
1 class CsvManager():
2     def __init__(self, file_path):
3         self.file_path = file_path
4         self.lock_path = self.file_path + ".lock"
5
6     def load_csv(self):
7         csv_header = pd.read_csv(self.file_path, nrows=0)
8         dates = [col for col in ["Date", "Created_At", "Last_Modified"] if col in csv_header.columns]
9         return pd.read_csv(self.file_path, index_col="Id", parse_dates=dates)
10
11
12     def save_csv(self, data_frame):
13         data_frame.to_csv(self.file_path, index=True)
14
15     def lock(self, timeout=10):
16         return FileLock(self.lock_path, timeout=timeout)
17
18     def generate_id(self, data_frame):
19         id_max = data_frame.index.max()
20         return id_max + 1 if not np.isnan(id_max) else 1
```

2.2. HistoryManager

Ce gestionnaire centralise le suivi de toutes les activités effectuées dans UtilityHub. Il permet de transformer chaque interaction (requête/réponse) en une donnée exploitable stockée dans **history.csv**.

Ses fonctions principales incluent :

- **L'enregistrement des actions** (`add_record`) : Chaque fois qu'un outil (Convertisseur, Downloader, etc.) termine une tâche, il communique avec ce manager pour laisser une trace de l'opération.
- **La récupération filtrée** (`get_history`) : Le manager est capable de renvoyer l'historique spécifique à chaque outil, permettant un affichage personnalisé dans les différentes fenêtres.
- **Le nettoyage et la suppression** (`delete_record`, `clear_history`) : Il offre des outils pour gérer l'espace et la confidentialité en supprimant des entrées individuelles ou en vidant tout l'historique d'un module.

```
1  BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
2  HISTORY_PATH = os.path.join(BASE_DIR, "data", "history.csv")
3
4  class HistoryManager():
5      def __init__(self, tool):
6          self.file_path = HISTORY_PATH
7          self.csv_manager = CsvManager(self.file_path)
8          self.tool = tool
9
10     def add_record(self, request, response):
11         with self.csv_manager.lock():
12             df = self.csv_manager.load_csv()
13
14             # the final file is playing the role of the id
15             cond = (df['Tool'] == self.tool) & (df['Response'] == response)
16
17             if any(cond):
18                 df.loc[cond, 'Date'] = datetime.now()
19                 df.loc[cond, 'Request'] = request
20             else:
21                 record_id = self.csv_manager.generate_id(df)
22
23                 current_datetime = datetime.now()
24                 df.loc[record_id] = [self.tool, request, response, current_datetime]
25
26             self.csv_manager.save_csv(df)
27
28
29     def get_history(self):
30         df = self.csv_manager.load_csv()
31         return df[df['Tool'] == self.tool]
```



```
1  def clear_history(self):
2      with self.csv_manager.lock():
3          df = self.csv_manager.load_csv()
4
5          df_new = df[df['Tool'] != self.tool]
6          self.csv_manager.save_csv(df_new)
7
8
9
10 def delete_record(self, id):
11     with self.csv_manager.lock():
12         df = self.csv_manager.load_csv()
13
14         df.drop(id, inplace=True, errors="ignore")
15         self.csv_manager.save_csv(df)
```

2.3. SettingsManager

Le point d'entrée unique pour la configuration de l'application. Contrairement aux autres managers, il travaille avec un fichier **JSON**, idéal pour stocker des préférences structurées.

Ses fonctions clés sont :

- **La lecture des paramètres** (`get_setting`) : Il permet de récupérer instantanément une valeur (ex: le dossier de destination des téléchargements ...).
- **La mise à jour dynamique** (`update_setting`) : Il sauvegarde immédiatement tout changement de paramètre dans **settings.json**, assurant que la configuration est persistante.
- **Le chargement global** (`get_all_settings`) : Utilisé principalement par l'interface de réglages pour afficher toutes les options disponibles en une seule fois.

```
1 # SETTINGS_PATH = r"utilityhub\data\settings.json" using this method
2 # result problems in working directory search
3
4 BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
5 SETTINGS_PATH = os.path.join(BASE_DIR, "data", "settings.json")
6
7 class SettingsManager:
8     def __init__(self):
9         self.settings_path = SETTINGS_PATH
10
11
12     def get_setting(self, category, key):
13         with open(self.settings_path, "r") as file:
14             data = json.load(file) # load for files, loads for string
15
16         return data[category][key]
17
18
19     def update_setting(self, category, key, value):
20         with open(self.settings_path, "r") as file:
21             data = json.load(file)
22
23         data[category][key] = value
24
25         with open(self.settings_path, "w") as file:
26             json.dump(data, file, indent=2)
27
28
29     def get_all_settings(self):
30         with open(self.settings_path, "r") as file:
31             return json.load(file)
32
```

3. Modules Fonctionnels

Chaque outil de UtilityHub a été développé comme un module indépendant, reposant sur une classe de service dédiée qui encapsule toute la logique métier.

3.1. Module Convertisseur

Ce module est le plus technique, s'appuyant sur plusieurs bibliothèques externes pour transformer une grande variété de formats.

- **Classe** : ConverterService
- **Décorateur** : `@refresh_settings` , Indispensable pour s'assurer que le chemin de destination (défini dans les réglages) est à jour avant chaque conversion.

```
1  from PIL import Image
2  import vtracer
3  from markdown import markdown
4  from xhtml2pdf import pisa
5  from core.history_manager import HistoryManager
6  from core.settings_manager import SettingsManager
7  import os
8  import subprocess
9
10 BASE_DIR = os.path.dirname(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
11 FFMPEG_PATH = os.path.join(BASE_DIR, "tools", "ffmpeg.exe")
12
13
14
15 def refresh_settings(func):
16     def wrapper(self, *args, **kwargs):
17         # "Before method"
18         self._refresh_settings()
19         result = func(self, *args, **kwargs)
20         # "After method"
21         return result
22     return wrapper
23
24
25
26 class ConverterService:
27
28     def __init__(self):
29         self.settingsManager = SettingsManager()
30         self.historyManager = HistoryManager("converter")
31
32         self.output_path = ""
33
34
35     def _refresh_settings(self):
36         self.output_path = self.settingsManager.get_setting("output_paths", "converter")
```

- Fonctions clés :
 - `convert_md_to_pdf` / `convert_txt_to_pdf` : Transforme des documents via markdown et xhtml2pdf.

```
1 @refresh_settings
2 def convert_md_to_pdf(self, input_file_path):
3     # Check input_file_path extensions
4     if not input_file_path.lower().endswith('.md'):
5         return -1
6
7
8     # Read the MD:
9     with open(input_file_path, "r", encoding="utf-8") as f:
10         md_text = f.read()
11
12     # md to html
13     html_content = markdown(md_text, extensions = ['fenced_code'])
14
15     # reminder: <pre>: Display this text exactly as written.
16     styled_html = f"""
17     <html>
18     <head>
19         <style>
20             body {{ font-family: Helvetica, Arial, sans-serif; font-size: 11pt; line-height: 1.5; color: #333; }}
21             h1, h2, h3 {{ color: #1f4068; border-bottom: 1px solid #1f4068; padding-bottom: 5px; }}
22             pre {{ background-color: #f4f4f4; padding: 10px; border-radius: 5px; white-space: pre-wrap; word-wrap: break-word; }}
23             code {{ font-family: Courier, monospace; color: #1f4068; background-color: #f7f7f7; padding: 2px 4px; border-radius: 3px; }}
24         </style>
25     </head>
26     <body>
27         {html_content}
28     </body>
29     </html>
30     """
31
32     output_file_name = os.path.basename(input_file_path).rsplit('.', 1)[0] + ".pdf"
33     output_file = os.path.join(self.output_path, output_file_name)
34
35     # convert html to pdf
36     with open(output_file, "wb") as file_out:
37         pisa.CreatePDF(styled_html, dest=file_out)
38
39     print("Markdown conversion complete!")
40
41     # Save history:
42     self.historyManager.add_record(input_file_path, output_file)
43     return output_file
44
```

```

1  @refresh_settings
2  def convert_txt_to_pdf(self, input_file_path):
3      if not input_file_path.lower().endswith('.txt'):
4          return -1
5
6      with open(input_file_path, "r", encoding="utf-8") as f:
7          text_content = f.read()
8
9      # Wrap in pre-tag for plain text preservation
10     html_content = f"""
11     <html>
12         <body>
13             <pre style='white-space: pre-wrap; font-family: Helvetica; font-size: 11pt;'>
14                 {text_content}
15             </pre>
16         </body>
17     </html>
18     """
19
20     output_file_name = os.path.basename(input_file_path).rsplit('.', 1)[0] + ".pdf"
21     output_file = os.path.join(self.output_path, output_file_name)
22
23     with open(output_file, "wb") as file_out:
24         pisa.CreatePDF(html_content, dest=file_out)
25
26     print("Text conversion complete! ")
27
28     # save history:
29     self.historyManager.add_record(input_file_path, output_file)
30     return output_file

```

- **convert_to_jpg / convert_to_png** : Gère le redimensionnement et la conversion d'images via Pillow.

```

1  @refresh_settings
2  def convert_to_jpg(self, input_file_path):
3      # Check input_file_path extensions
4      valid_ext = (".png", ".webp", ".bmp", ".tiff")
5      if not input_file_path.lower().endswith(valid_ext):
6          return -1
7
8
9      output_file_name = os.path.basename(input_file_path).rsplit('.', 1)[0] + ".jpg"
10     output_file = os.path.join(self.output_path, output_file_name)
11
12     with Image.open(input_file_path) as img:
13         # Convert RGBA (transparency) to RGB (flat color)
14
15         if img.mode in ("RGBA", "P"):
16             img = img.convert("RGB")
17         img.save(output_file, "JPEG", quality=95) # .rsplit(separator, maxsplit)
18
19
20     print("jpg conversion complete! ")
21
22     # Save history:
23     self.historyManager.add_record(input_file_path, output_file)
24
25     return output_file
26
27
28
29 @refresh_settings
30 def convert_to_png(self, input_file_path):
31     # Check extensions:
32     valid_ext = (".jpg", ".jpeg", ".webp", ".bmp")
33     if not input_file_path.lower().endswith(valid_ext):
34         return -1
35
36     output_file_name = os.path.basename(input_file_path).rsplit('.', 1)[0] + ".png"
37     output_file = os.path.join(self.output_path, output_file_name)
38
39     with Image.open(input_file_path) as img:
40         img.save(output_file, "PNG")
41
42     print("png conversion complete! ")
43     # Save history:
44     self.historyManager.add_record(input_file_path, output_file)
45
46     return output_file

```

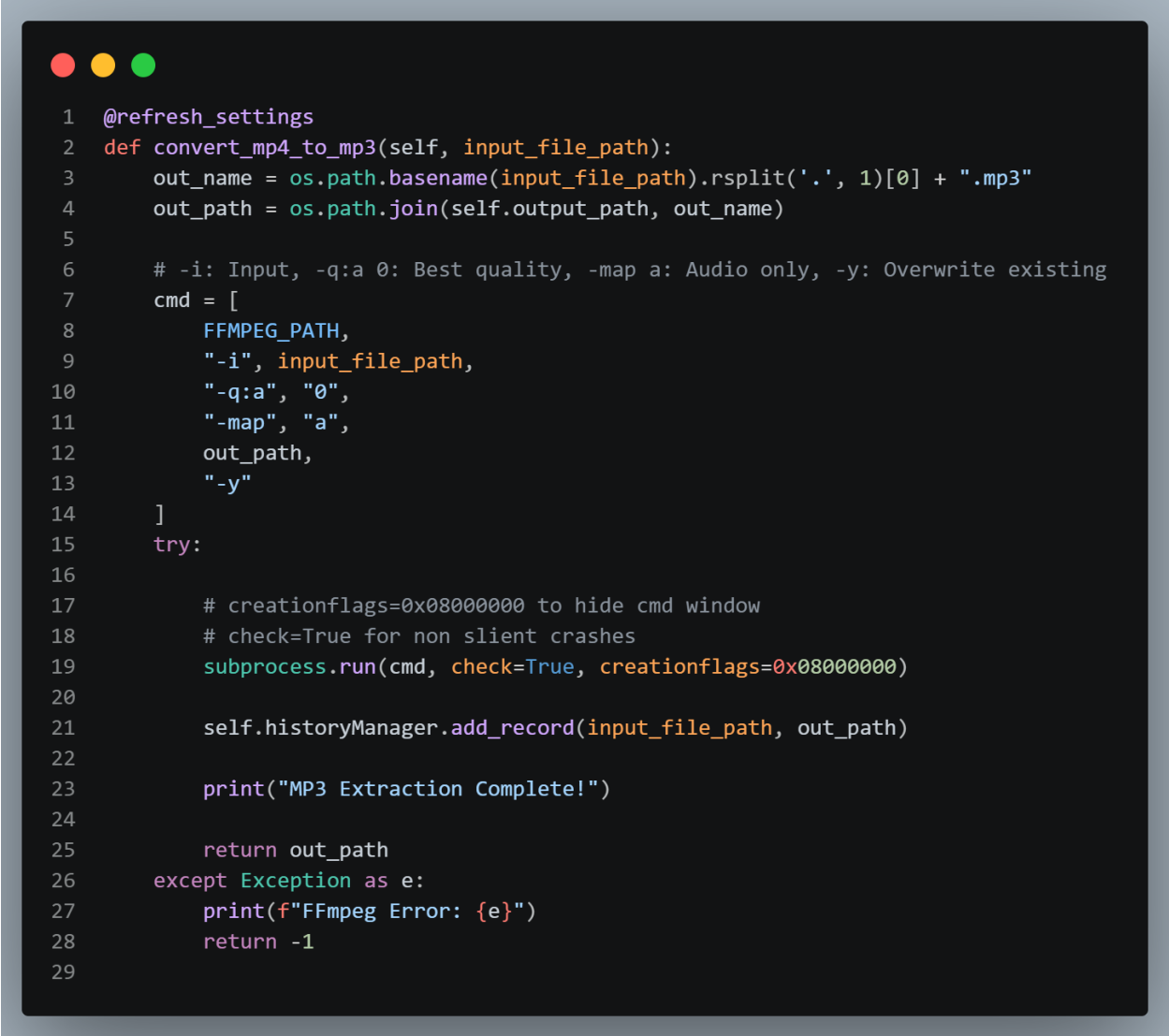
- **convert_to_svg** : Utilise vtracer pour vectoriser une image bitmap.

```

1
2 @refresh_settings
3 def convert_to_svg(self, input_file_path):
4     # Check extensions:
5     valid_ext =(".jpg", ".jpeg", ".png", ".webp")
6     if not input_file_path.lower().endswith(valid_ext):
7         return -1
8
9     output_file_name = os.path.basename(input_file_path).rsplit('.', 1)[0] + ".svg"
10    output_file = os.path.join(self.output_path, output_file_name)
11
12    try:
13        # It converts pixels into mathematical SVG paths
14        vtracer.convert_image_to_svg_py(
15            input_file_path, # Input Path
16            output_file,     # Output Path
17            "color",         # Colormode (color vs binary)
18            "stacked",       # Hierarchical (stacked vs cutout)
19            "spline",        # Mode (spline, polygon, none)
20            4,               # Filter Speckle
21            6,               # Color Precision
22            60,              # Corner Threshold
23            4,               # Gradient Step
24            2,               # Path Precision
25            10,              # Max Iterations
26            45,              # Splice Threshold
27            10               # Segment Length
28        )
29    except Exception as e:
30        print(f"vtracer exception {e}")
31
32
33
34    print("svg conversion complete! ")
35
36    # Save history:
37    self.historyManager.add_record(input_file_path, output_file)
38
39    return output_file

```

- **convert_mp4_to_mp3** : Pilote le binaire FFmpeg via un sous-processus pour extraire l'audio d'une vidéo.



```

1  @refresh_settings
2  def convert_mp4_to_mp3(self, input_file_path):
3      out_name = os.path.basename(input_file_path).rsplit('.', 1)[0] + ".mp3"
4      out_path = os.path.join(self.output_path, out_name)
5
6      # -i: Input, -q:a 0: Best quality, -map a: Audio only, -y: Overwrite existing
7      cmd = [
8          FFMPEG_PATH,
9          "-i", input_file_path,
10         "-q:a", "0",
11         "-map", "a",
12         out_path,
13         "-y"
14     ]
15     try:
16
17         # creationflags=0x08000000 to hide cmd window
18         # check=True for non silent crashes
19         subprocess.run(cmd, check=True, creationflags=0x08000000)
20
21         self.historyManager.add_record(input_file_path, out_path)
22
23         print("MP3 Extraction Complete!")
24
25         return out_path
26     except Exception as e:
27         print(f"FFmpeg Error: {e}")
28         return -1
29

```

3.2. Module Téléchargeur (Downloader)

Il gère l'extraction et le téléchargement de médias depuis des plateformes web complexes.

- **Classe** : MediaDownloaderService
- **Décorateur** : @refresh_settings – Recharge le dossier de téléchargement et les préférences de navigateur (cookies).

```

1 import yt_dlp
2 from core.history_manager import HistoryManager
3 from core.settings_manager import SettingsManager
4 import os
5 BASE_DIR = os.path.dirname(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
6 FFmpeg_PATH = os.path.join(BASE_DIR, "tools", "ffmpeg.exe")
7
8
9
10 def refresh_settings(func):
11     def wrapper(self, *args, **kwargs):
12         # "Before method"
13         self._refresh_settings()
14         result = func(self, *args, **kwargs)
15         # "After method"
16         return result
17     return wrapper
18
19
20 class MediaDownloaderService:
21     def __init__(self):
22         self.settingsManager = SettingsManager()
23         self.historyManager = HistoryManager("downloader")
24
25         self.output_path = ""
26         self.cookies_browser = "firefox"
27
28
29     def _refresh_settings(self):
30         self.output_path = self.settingsManager.get_setting("output_paths", "downloader")
31         try:
32             self.cookies_browser = self.settingsManager.get_setting("preferences", "downloader_browser")
33         except KeyError:
34             self.cookies_browser = "firefox"

```

- Fonctions clés :
 - `get_media_info` : Récupère les métadonnées (titre, miniature, formats disponibles) via yt-dlp.



```
1  @refresh_settings
2  def get_media_info(self, url):
3      try:
4          ydl_opts = {
5              "quiet": True,
6              'no_warnings': True,
7              "noplaylist": True,
8              'restrictfilenames': True,
9              'cookiesfrombrowser': (self.cookies_browser,),
10             'verbose': True,
11             'extract_flat': False,
12             'nocheckcertificate': True,
13         }
14
15         with yt_dlp.YoutubeDL(ydl_opts) as ydl:
16             info = ydl.extract_info(url, download=False)
17             return {
18                 "metadata": {
19                     "title": info.get("title", "Unknown Title"),
20                     "duration": info.get("duration", 0),
21                     "thumbnail": info.get("thumbnail"),
22                     "uploader": info.get("uploader", "Unknown")
23                 },
24                 "formats": self.get_available_formats(info)
25             }
26
27     except:
28         # Error: try again (maybe url not functioning)
29         return None
```

- **download_media** : Lance le téléchargement effectif avec un progress_hook pour un suivi en temps réel.

```

1  @refresh_settings
2  def download_media(self, url, format_choice, progress_hook, is_audio=False):
3      """
4      In yt_dlp, a progress_hook is a function that gets called repeatedly
5      while the download is running. It lets you track progress
6      (percentage, speed, ETA, etc.) in real time.
7      """
8      output_filename = None
9
10
11     try:
12         if is_audio:
13             # Try specific choice, then any best audio
14             final_format = f"{format_choice}/bestaudio/best"
15
16         else:
17             # try specific choice + best audio
18             # try specific choice alone (combined or no audio)
19             # try overall best video + best audio
20             # try any 'best' format
21             final_format = f"({format_choice}+bestaudio)/({format_choice})/bestvideo+bestaudio/best"
22
23
24
25     ydl_opts = {
26         'ffmpeg_location': FFMPEG_PATH,
27         "format": final_format, # (format_id)
28         "outtmpl": f"{self.output_path}/{%(title)s}-%(ext)s.%(ext)s",
29         "progress_hooks": [progress_hook],
30         "merge_output_format": "mp4",
31         """
32         # tells yt-dlp:
33         If you have to stitch multiple files together (like HD video + audio),
34         please put the final result inside an mp4 container.
35         """
36
37         "noplaylist": True, # Ensure it doesn't try to fetch a whole playlist
38         'restrictfilenames': True,
39         'cookiesfrombrowser': (self.cookies_browser,),
40         'nocheckcertificate': True,
41     }
42
43     with yt_dlp.YoutubeDL(ydl_opts) as ydl:
44         ydl.download([url])
45
46         info = ydl.extract_info(url, download=False)
47
48         output_filename = ydl.prepare_filename(info)
49
50         # Save history:
51         self.historyManager.add_record(url, output_filename )
52
53         return "SUCCESS", output_filename
54
55
56     except Exception as e:
57         return "ERROR", output_filename # to delete the file
58

```

- **get_available_formats** : Analyse les données brutes pour proposer des choix clairs (résolution, extension) à l'utilisateur.

```
1 def get_available_formats(self, info):
2
3     formats = []
4     for format in info.get("formats",[]):
5
6         vcodec = format.get("vcodec")
7         acodec = format.get("acodec")
8
9         is_audio = (vcodec == "none" or vcodec is None)
10
11
12     # "vcodec" (video codec) or "acodec" (audio codec)
13     if vcodec != "none" or acodec != "none":
14         formats.append({
15             "format_id": format.get("format_id"),
16             "ext": format.get("ext"),
17             "resolution": "Audio" if is_audio else (format.get("format_note") or format.get("resolution") or "Video")
18         })
19
20     return formats
21
```

3.3. Module Notes

Un système d'archivage textuel léger et rapide.

- **Classe** : NotesService



```
1 BASE_DIR = os.path.dirname(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
2 NOTESPATH = os.path.join(BASE_DIR, "data", "notes.csv")
3
4 class NotesService:
5     def __init__(self):
6         self.csvmanager = CsvManager(NOTESPATH)
7         self.history_manager = HistoryManager("notes")
```

- **Fonctions clés :**

- **save_note** : Gère à la fois la création d'une nouvelle note et la mise à jour d'une note existante (Edit).



```
1 def save_note(self, title, content, Tag="", id = None):
2     with self.csvmanager.lock():
3         df = self.csvmanager.load_csv()
4         if id is None:
5             record_id = self.csvmanager.generate_id(df)
6             created_at = datetime.now()
7         else:
8             record_id = id
9             created_at = df.loc[id]['Created_At']
10
11
12     df.loc[record_id] = [title, content, created_at, datetime.now(), Tag] # iloc doesn't allow creation
13     self.csvmanager.save_csv(df)
14
15
16     # save history:
17     self.history_manager.add_record(f"Create/Edit Note {title}", f"Saved Note #{record_id}")
18     return record_id
19
```

- **list_notes / get_note_by_id** : Lecture globale ou ciblée des données.



```
1 def list_notes(self):
2     return self.csvmanager.load_csv()
3
4
5 def get_note_by_id(self, id):
6     df = self.csvmanager.load_csv()
7     return df.loc[id].to_dict()
```

- **search_notes / search_by_tag** : Système de filtrage avancé pour retrouver rapidement une information parmi des centaines de notes.



```
1 def search_notes(self, keyword):
2     df = self.csvmanager.load_csv()
3     # na=False -> contains treats na as False
4     cond = df['Title'].str.contains(keyword, case=False, na=False) | df["Content"].str.contains(keyword, case=False, na=False)
5     search_df = df[ cond ]
6     return search_df
7
8
9
10 def search_by_tag(self, tag_key):
11     df = self.csvmanager.load_csv()
12     df = df.sort_values(by="Last_Modified", ascending=False)
13
14
15     if tag_key == "All":
16         return df
17
18     cond = df["Tag"] == tag_key # because UI support only 1 tag
19     return df[ cond ]
20
```

- **change_Tag / change_Title / change_Content** : Modifications granulaires des métadonnées ou du contenu.

```

1  def change_Tag(self, id, new_tag):
2      with self.csvmanager.lock():
3          df = self.csvmanager.load_csv()
4          if (id in df.index):
5              df.loc[id, ["Last_Modified", "Tag"]] = [datetime.now(), new_tag]
6              self.csvmanager.save_csv(df)
7
8              self.history_manager.add_record("Modify Note", f"Changed Tag on Note #{id}")
9
10             return True
11         else:
12             return False
13
14  def change_Title(self, id, new_title):
15      with self.csvmanager.lock():
16          df = self.csvmanager.load_csv()
17          if (id in df.index):
18              df.loc[id, ["Last_Modified", "Title"]] = [datetime.now(), new_title]
19              self.csvmanager.save_csv(df)
20
21              self.history_manager.add_record("Modify Note", f"Changed title on Note #{id}")
22
23             return True
24         else:
25             return False
26
27  def change_Content(self, id, new_content):
28      with self.csvmanager.lock():
29          df = self.csvmanager.load_csv()
30          if (id in df.index):
31              df.loc[id, ["Last_Modified", "Content"]] = [datetime.now(), new_content]
32              self.csvmanager.save_csv(df)
33
34              self.history_manager.add_record("Modify Note", f"Changed Content on Note #{id}")
35             return True
36         else:
37             return False

```

- **delete_note** : Suppression sécurisée avec mise à jour immédiate du fichier CSV.



```
1 def delete_note(self, id):
2     with self.csvmanager.lock():
3         df = self.csvmanager.load_csv()
4         if (id in df.index) :
5             df.drop(id, inplace = True, errors="ignore")
6             self.csvmanager.save_csv(df)
7
8             # save history:
9             self.history_manager.add_record("Delete Note", f"Deleted Note #{id}")
10
11         return True
12     else:
13         return False
```

3.4. Module Todo (Tâches)

Conçu pour la gestion de listes de tâches avec suivi d'état.

- **Classe** : TodoService

```

1 from core.csv_manager import CsvManager
2 from core.history_manager import HistoryManager
3 from datetime import datetime
4 import os
5
6 BASE_DIR = os.path.dirname(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
7 TODOPATH = os.path.join(BASE_DIR, "data", "todos.csv")
8
9 class TodoService:
10     def __init__(self):
11         self.csvmanager = CsvManager(TODOPATH)
12         self.history_manager = HistoryManager("todo")
13

```

- Fonctions clés :
 - add_note / delete_task : Gestion de l'ajout et de la suppression.

```

1 def add_note(self, task):
2     with self.csvmanager.lock():
3         df = self.csvmanager.load_csv()
4         record_id = self.csvmanager.generate_id(df)
5         df.loc[record_id] = [task, "Pending", datetime.now()] # iloc doesn't allow creation
6         self.csvmanager.save_csv(df)
7
8         # save history:
9         self.history_manager.add_record(f"Create Task", f"Created Task #{record_id}")
10        return record_id
11
12
13 def delete_task(self, id):
14     with self.csvmanager.lock():
15         df = self.csvmanager.load_csv()
16         if (id in df.index) :
17             df.drop(id, inplace = True, errors="ignore")
18             self.csvmanager.save_csv(df)
19
20             # save history:
21             self.history_manager.add_record("Delete Task", f"Deleted Task #{id}")
22
23             return True
24         else:
25             return False

```

- change_status / change_task : Permet de basculer l'état (Pending, Done) ou de modifier le texte.



```

1  def change_status(self, id, new_status):
2      with self.csvmanager.lock():
3          df = self.csvmanager.load_csv()
4          if (id in df.index):
5              df.loc[id, "Status"] = new_status
6              self.csvmanager.save_csv(df)
7
8              self.history_manager.add_record("Modify Task", f"Changed status on Task #{id}")
9
10             return True
11         else:
12             return False
13
14
15
16  def change_task(self, id, new_task):
17      with self.csvmanager.lock():
18          df = self.csvmanager.load_csv()
19          if (id in df.index):
20              df.loc[id, "Task"] = new_task
21              self.csvmanager.save_csv(df)
22
23              self.history_manager.add_record("Modify Task", f"Changed task on Task #{id}")
24
25             return True
26         else:
27             return False

```

- **get_tasks** : Récupération triée par date



```

1  def get_tasks(self):
2      df = self.csvmanager.load_csv()
3
4      df_new = df.sort_values("Date", ascending=False)
5      return df_new

```

- **search_tasks** / **search_by_status** : Filtrage pour se concentrer sur les priorités du moment.



```
1  def search_tasks(self, keyword):
2      df = self.csvmanager.load_csv()
3      # na=False -> contains treats na as False
4      cond = df['Task'].str.contains(keyword, case=False, na=False)
5
6      search_df = df[ cond ].sort_values("Date", ascending=False)
7      return search_df
8
9
10
11
12 def search_by_status(self, status):
13     df = self.csvmanager.load_csv()
14     cond = df["Status"] == status # because UI support only 1 status
15
16     search_df = df[ cond ].sort_values("Date", ascending=False)
17     return search_df
18
```

3.5. Module Historique

Contrairement aux autres modules, il ne possède pas de service propre mais utilise directement le HistoryManager de la couche Core.

- **Rôle** : Servir d'interface de consultation pour les journaux d'activité.

- **Opérations** : Permet au Desktop et au Web d'afficher un tableau récapitulatif des actions passées, d'ouvrir les fichiers générés, ou de nettoyer les traces obsolètes via les méthodes du manager.

Mise en œuvre Technique

1. Interface Desktop (PyQt5)

L'interface logicielle de **UtilityHub** a été développée avec **PyQt5**, en mettant l'accent sur l'ergonomie et la séparation des tâches.

1.1. Navigation et Menu principal

L'application utilise un tableau de bord central qui gère tout le menu. On peut passer d'un outil (Convertisseur, Téléchargeur, etc.) à l'autre très facilement sans jamais fermer ou recharger l'application. Cela permet de garder son travail en cours même si on change d'onglet.

1.2. Rapidité et Travail en arrière-plan

Pour éviter que la fenêtre ne se bloque (ne freeze) pendant une tâche longue (comme un gros téléchargement), nous utilisons des tâches de fond (threads) :

- Les tâches lourdes sont lancées en arrière-plan pour que l'interface reste toujours fluide et réactive.
- L'outil communique sa progression en direct (pourcentage, vitesse) vers l'écran en utilisant des signaux, ce qui permet de mettre à jour les barres de progression en temps réel.

1.3. Le défi du GIL (Global Interpreter Lock)

Il y a une différence technique entre les outils :

- **Téléchargement** : Comme c'est surtout de l'attente internet, l'interface reste très fluide même avec plusieurs téléchargements.
- **Convertisseur** : Lors d'une conversion, l'interface a tendance à se bloquer à cause du **GIL** de Python. Bien que nous utilisions des outils externes (FFmpeg), les ressources demandées par ces tâches finissent par bloquer l'application. L'utilisation d'**asyncio** aurait été la solution idéale pour assurer une fluidité totale, mais je n'ai pas eu le temps de l'ajouter dans cette version.

2. Interface Web (Flask)

La version Web de UtilityHub transpose ces concepts dans un environnement distribué accessible via navigateur.

2.1. Organisation par modules

Le site web est découpé en plusieurs morceaux indépendants (appelés "Blueprints"). C'est comme avoir plusieurs petites applications qui travaillent ensemble pour former le site. Cela rend le code plus propre et beaucoup plus facile à modifier ou à faire évoluer.

2.2. Suivi en direct du téléchargement

L'un des plus gros défis a été d'afficher la progression du téléchargement sur le web. Normalement, un site attend la fin d'une tâche avant d'afficher un résultat.

- Pour régler ça, nous avons utilisé un système de flux en direct appelé **SSE (Server-Sent Events)**.

2.2.1. Détails techniques du SSE :

Le SSE maintient une connexion HTTP persistante ouverte afin que le serveur puisse envoyer des données au client en temps réel sans avoir à rafraîchir la page (polling). Le client se connecte une seule fois via l'API `EventSource`, et le serveur transmet des événements en texte brut formatés sous la forme `data: ... \n\n` avec l'en-tête `Content-Type: text/event-stream`.

Avantages clés : Plus simple que les WebSockets (flux unidirectionnel), reconnexion automatique en cas de coupure, et compatibilité avec les pare-feu puisqu'il utilise les ports HTTP standards.

Usage dans UtilityHub : Il est utilisé pour diffuser la progression du téléchargement (pourcentage, vitesse, estimation du temps restant) depuis le processus d'arrière-plan vers l'interface Web en temps réel.

2.3. Un design moderne et dynamique

L'interface web utilise **Bootstrap** pour être élégante et s'adapter à tous les écrans (ordinateurs, tablettes). Les données (historique, notes, tâches) sont récupérées automatiquement depuis nos fichiers et affichées proprement dans le navigateur.

3. Synchronisation et Sécurité (FileLock)

Le système **FileLock** est ce qui permet au logiciel Desktop et au site Web de travailler ensemble sans se gêner. Comme les deux versions partagent les mêmes fichiers de données (notes, tâches), il y avait un risque qu'elles s'écrasent mutuellement si elles écrivaient au même moment. Le FileLock met un "verrou" de sécurité : dès qu'une partie commence à écrire, l'autre attend son tour, garantissant ainsi que les données ne sont jamais perdues ou cassées.

4. Pourquoi le Desktop est plus avancé ?

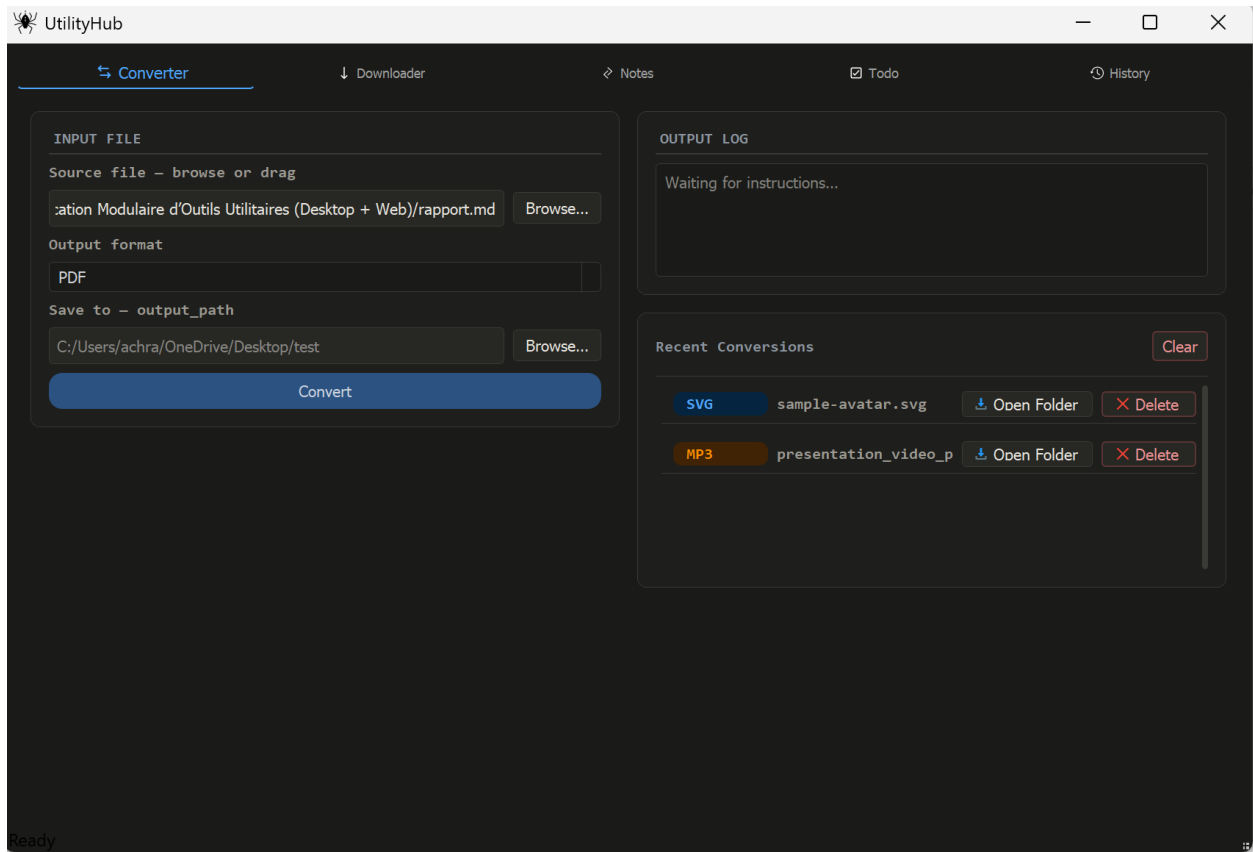
J'ai passé beaucoup plus de temps sur l'application de bureau (PyQt5). C'est pourquoi elle est plus belle et propose plus d'options (comme la possibilité d'annuler une tâche en cours) que la version Web pour le moment.

Présentation Graphique

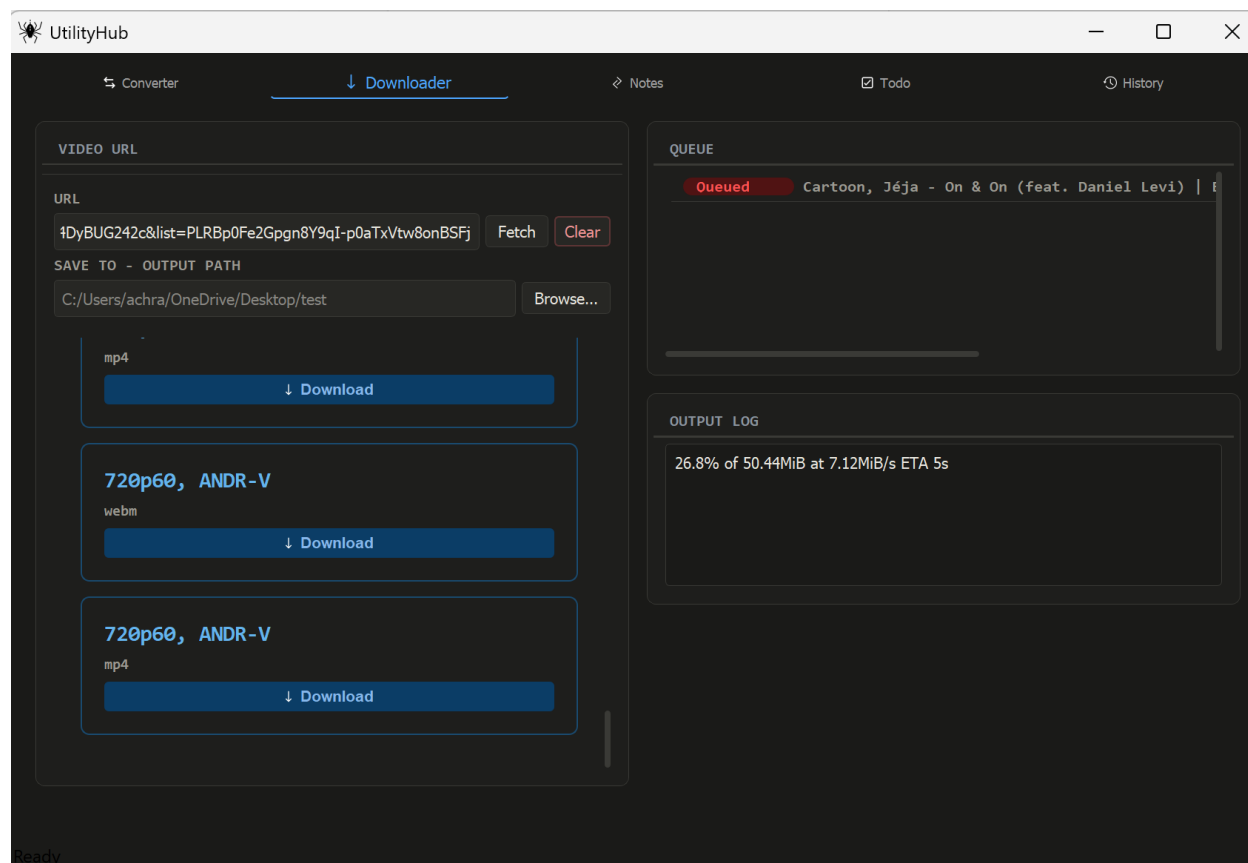
Cette section présente l'interface utilisateur de **UtilityHub** à travers ses différents modules.

1. Interface Desktop (PyQt5)

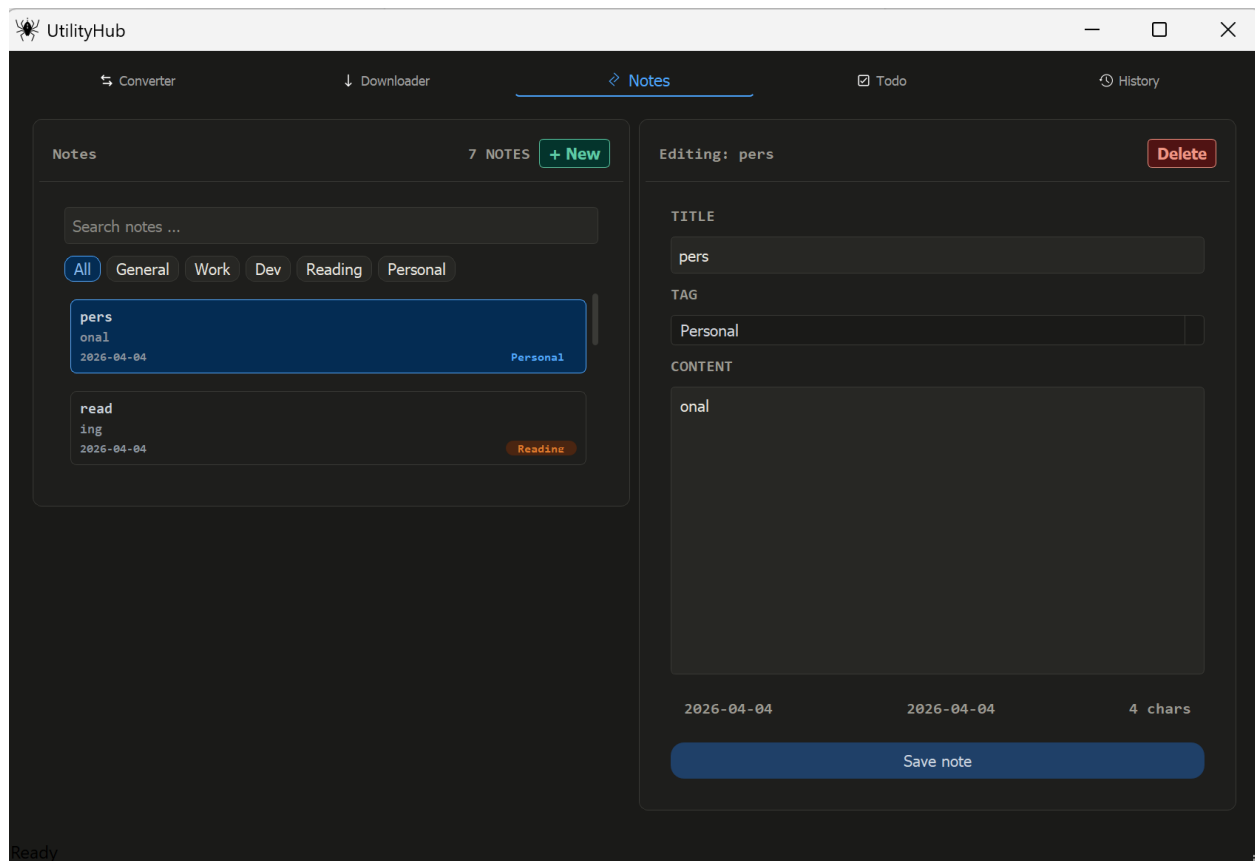
1.1. Module Convertisseur



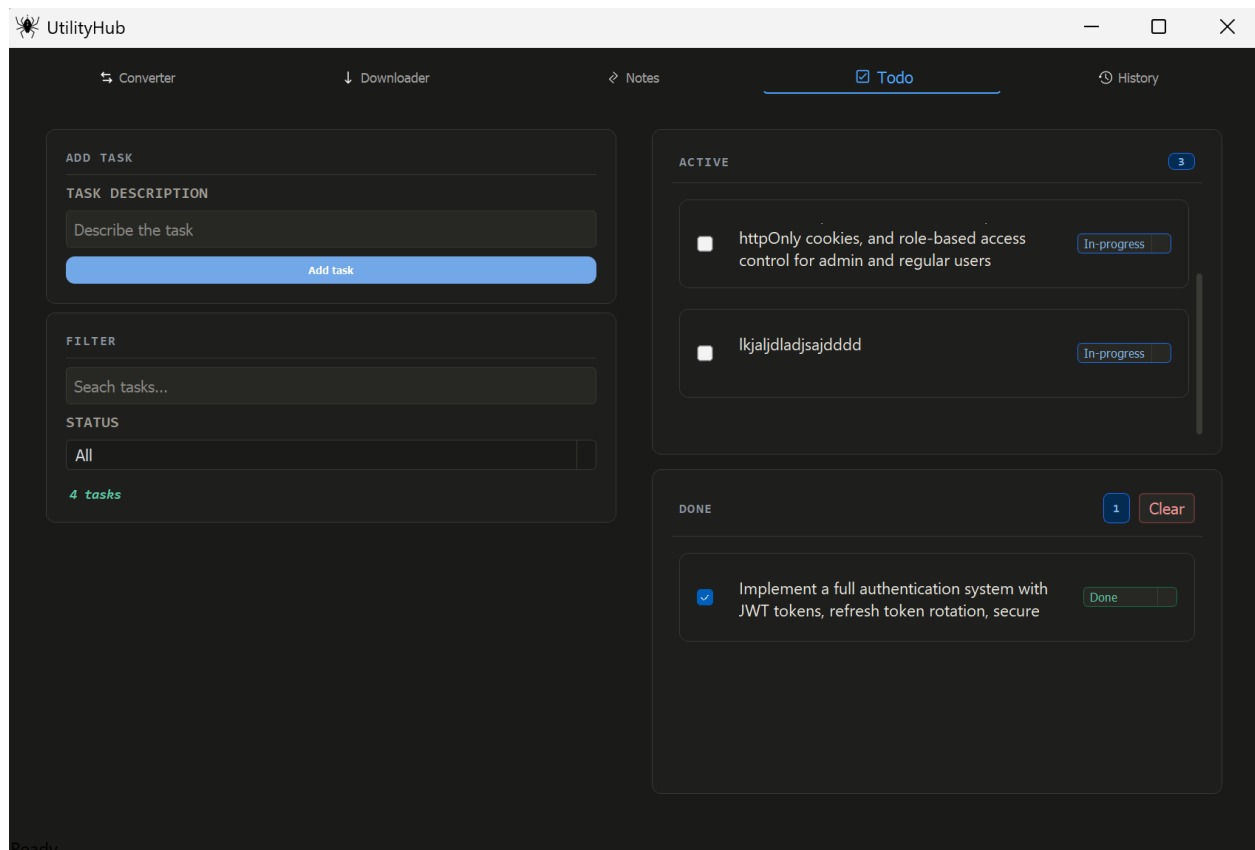
1.2. Module Téléchargeur



1.3. Gestion des Notes



1.4. Liste de tâches



1.5. Historique des activités

UtilityHub

Converter

Downloader

Notes

Todo

History

ACTION HISTORY

Clear all

Tool	Request	Response	DATETIME	
todo	Create Task	Created Task #6	2026-04-04 19:52	×
todo	Modify Task	Changed status on Ta..	2026-04-04 19:52	×
todo	Modify Task	Changed status on Ta..	2026-04-04 19:52	×
todo	Modify Task	Changed status on Ta..	2026-04-10 09:55	×
notes	Create/Edit Note ho	Saved Note #2	2026-04-04 19:53	×
notes	Create/Edit Note rea..	Saved Note #8	2026-04-04 19:53	×

2. Interface Web (Flask)

2.1. Page d'accueil / Convertisseur

 UtilityHub Converter Downloader Notes Todo History

File Converter



Drag & drop or click to select
.md .txt .png .jpg .jpeg .webp .bmp .tiff

Browse files

OUTPUT FORMAT

PNG

Convert

2.2. Page du Téléchargeur

 UtilityHub Converter Downloader Notes Todo History

Video Downloader

https://youtube.com/watch?v=...

Fetch

OUTPUT LOG

13.6% of 319.98MiB at 3.32MiB/s ETA 83s

Available Formats

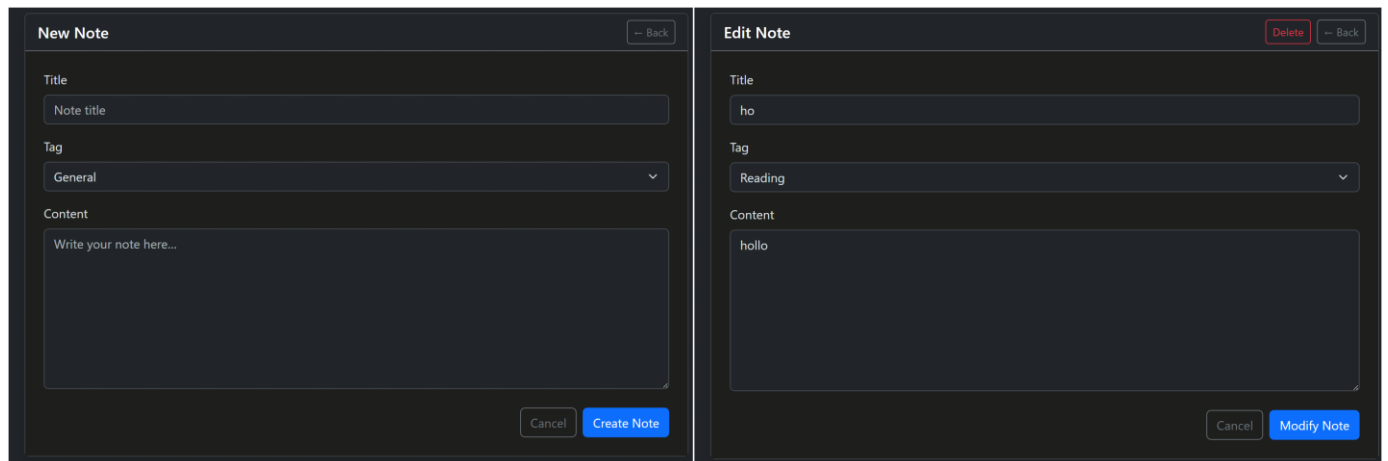
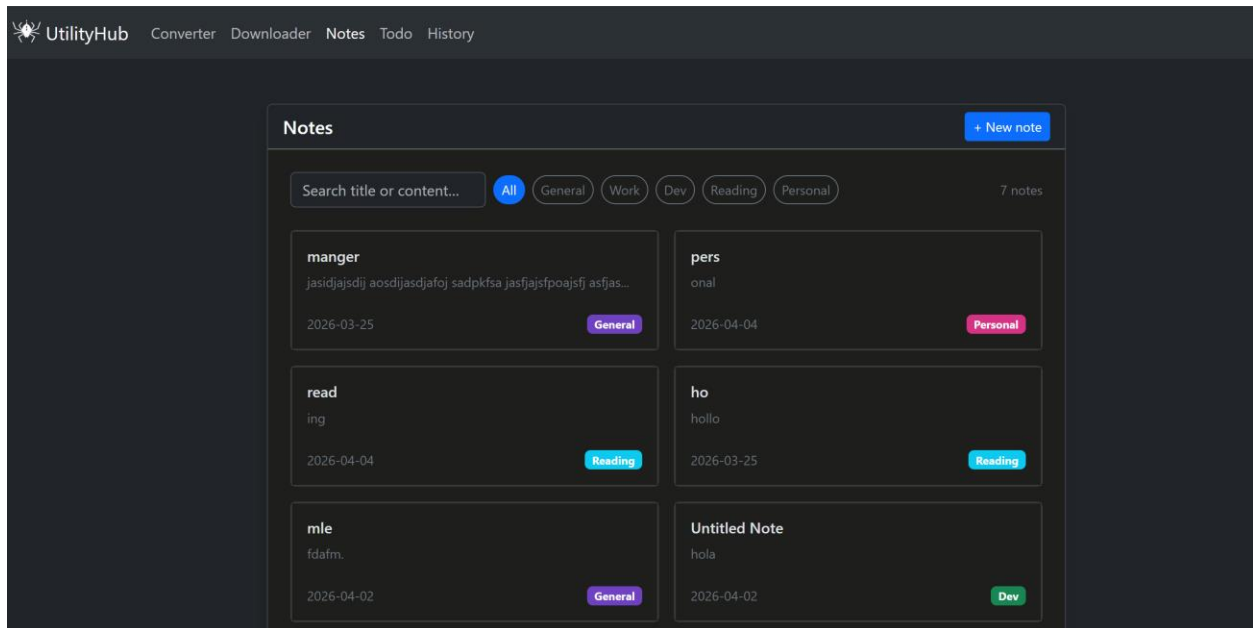
New URL

gabriawll - Retake | House | NCS - Copyright Free Music

NoCopyrightSounds · 2:26

Audio m4a Download	Audio webm Download	Audio m4a Download	Audio webm Download
144p, ANDR-V mp4 Download	144p, ANDR-V webm Download	144p, ANDR-V mp4 Download	240p, ANDR-V mp4 Download

2.3. Page des Notes



2.4. Liste des Tâches

UtilityHub

ConverterDownloaderNotesTodoHistory

Todo

Search title or content...

AllPendingIn ProgressDone

3 tasks

Describe the task

Add Task

Active2

☐ Implement a full authentication system with JWT tokens, refresh token rotation, secure httpOnly cookies, and role-based access control for admin and regular users

In-progress

2026-04-04 14:16:49.144487

☐ lkjaljldadsajddddd

In-progress

2026-04-04 14:15:23.447327

Done1Clear All

☒ Implement a full authentication system with JWT tokens, refresh token rotation, secure httpOnly cookies, and role-based access control for admin and regular users

Done

2026-04-04 19:52:17.250201

2.5. Historique

All actions Clear all			
SERVICE	REQUEST	RESPONSE	DATETIME
notes	Edit Note manger	Saved Note #4	2026-04-13 23:05
todo	Modify Task	Changed status on Task #6	2026-04-13 23:03
todo	Modify Task	Changed status on Task #7	2026-04-13 23:03
todo	Delete Task	Deleted Task #7	2026-04-13 23:03
todo	Create Task	Created Task #7	2026-04-10 10:09
converter	presentation_video_pyqt.mp4	presentation_video_pyqt.mp3	2026-04-10 10:02
converter	sample-avatar.jpg	sample-avatar.svg	2026-04-10 09:58
todo	Modify Task	Changed status on Task #2	2026-04-10 09:55
notes	Edit Note ho	Saved Note #2	2026-04-04 19:53
notes	Edit Note read	Saved Note #8	2026-04-04 19:53
notes	Edit Note pers	Saved Note #9	2026-04-04 19:53
todo	Create Task	Created Task #6	2026-04-04 19:52
todo	Modify Task	Changed status on Task #5	2026-04-04 19:52

Conclusion

La réalisation du projet **UtilityHub** a constitué une expérience d'ingénierie logicielle complète, alliant développement applicatif desktop et architecture web. L'objectif initial – concevoir un hub d'outils utilitaires centralisé et modulaire – a été atteint en proposant une solution fonctionnelle capable de manipuler des données hétérogènes (médias, documents, notes) de manière fluide.

Sur le plan technique, ce projet a permis de répondre à des problématiques concrètes de développement :

- **La modularité** : L'architecture en couches (Core, Services, Présentation) garantit une séparation nette des responsabilités, rendant l'application facilement évolutive.
- **L'hybridité** : L'intégration réussie de **PyQt5** et **Flask** démontre la versatilité de Python. La gestion des threads pour l'interface desktop et l'implémentation des Server-Sent Events (SSE) pour le web ont permis d'offrir une expérience utilisateur réactive malgré la complexité de certaines tâches.
- **La synchronisation** : L'utilisation de **FileLock** a assuré l'intégrité des données lors d'accès concurrents entre les deux interfaces, garantissant une persistance fiable via des fichiers CSV gérés par **Pandas**.

Bien que des défis subsistent, notamment la gestion totale du "blocking" lors des tâches de conversion intensives (lié aux limitations du GIL), des solutions futures comme l'implémentation native d'**asyncio** ont été identifiées pour perfectionner la fluidité du système.

En conclusion, **UtilityHub** dépasse le simple cadre d'un agrégateur d'outils pour devenir une plateforme solide, capable de piloter des binaires externes complexes (FFmpeg) et de centraliser la productivité. Ce projet constitue une base robuste et évolutive, prête à accueillir de nouveaux modules à l'avenir.